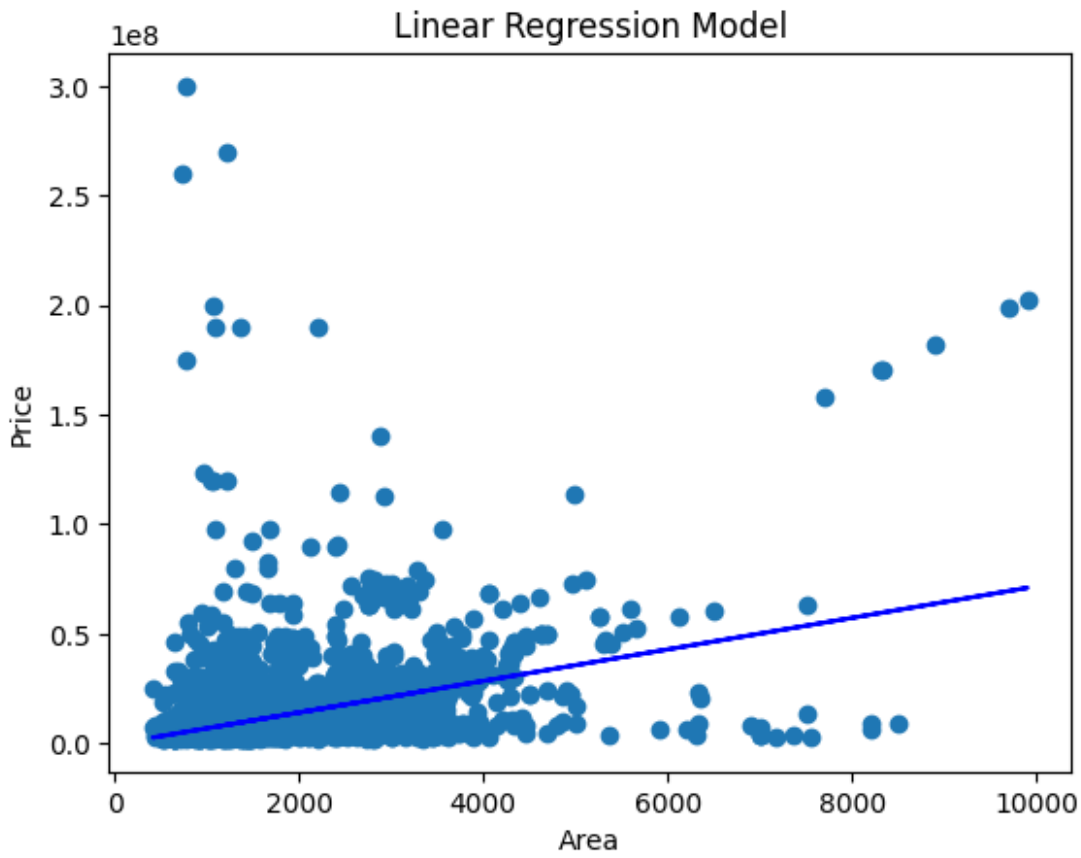
```python
# EDA
import pandas as pd

df = pd.read_csv("/content/sample_data/Bangalore.csv")


print(df.isnull().sum())
```

```
Price                  0
Area                   0
Location               0
No. of Bedrooms        0
Resale                 0
MaintenanceStaff       0
Gymnasium              0
SwimmingPool           0
LandscapedGardens      0
JoggingTrack           0
RainWaterHarvesting    0
IndoorGames            0
ShoppingMall           0
Intercom               0
SportsFacility         0
ATM                    0
ClubHouse              0
School                 0
24X7Security           0
PowerBackup            0
CarParking             0
StaffQuarter           0
Cafeteria              0
MultipurposeRoom       0
Hospital               0
WashingMachine         0
Gasconnection          0
AC                     0
Wifi                   0
Children'splayarea     0
LiftAvailable          0
BED                    0
VaastuCompliant        0
Microwave              0
GolfCourse             0
TV                     0
DiningTable            0
Sofa                   0
Wardrobe               0
Stadium                0
dtype: int64
```

Task1: Question 1:

Build a Simple Linear Regression model to predict the Sale price of the house. Use Area as the independent variable

```
import matplotlib.pyplot as plt
from sklearn.linear_model import  LinearRegression
from sklearn.model_selection import train_test_split

X = df[['Area']]
y = df['Price']

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

plt.scatter(X,y)
plt.plot(X, model.predict(X), color='blue')
plt.xlabel('Area')
plt.ylabel('Price')
plt.title('Linear Regression Model')


Text(0.5, 1.0, 'Linear Regression Model')
```



Question 2 :Build Multiple Linear Regression model to predict Sale price of the house.

```
import matplotlib.pyplot as plt
from sklearn.linear_model import  LinearRegression
from sklearn.model_selection import train_test_split

X = df[['Area','No. of Bedrooms', 'Location']]
y = df['Price']
X = pd.get_dummies(X, columns=['Location'], drop_first=True)


X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(y_pred)
```

```
[15219941.14102224  6739144.83392015  6183317.14954343 ...
  7172158.2858515   7237487.26194814  8783252.54745754]
```

Question 3: Build a model using Lasso and Ridge regression to reduce model complexity.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Lasso, Ridge
from sklearn.metrics import mean_squared_error, r2_score

X = df[['Area','No. of Bedrooms', 'Location']]
y = df['Price']
X = pd.get_dummies(X, columns=['Location'], drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

Linear_model = LinearRegression()
Linear_model.fit(X_train, y_train)

y_pred = Linear_model.predict(X_test)

## Lasso
L1 = Lasso(alpha=0.1)
L1.fit(X_train, y_train)

y_pred_L1 = L1.predict(X_test)

## Ridge
L2 = Ridge(alpha = 1.0)
L2.fit(X_train, y_train)

y_pred_L2 = L2.predict(X_test)

print("------- Linear Regression -------")
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2:", r2_score(y_test, y_pred))

print("------- Lasso -------")
print("MSE:", mean_squared_error(y_test, y_pred_L1))
print("R2:", r2_score(y_test, y_pred_L1))

print("------- Ridge -------")
print("MSE:", mean_squared_error(y_test, y_pred_L2))
print("R2:", r2_score(y_test, y_pred_L2))
```

Question 4: Build an SVR model to predict Sale price of the house.

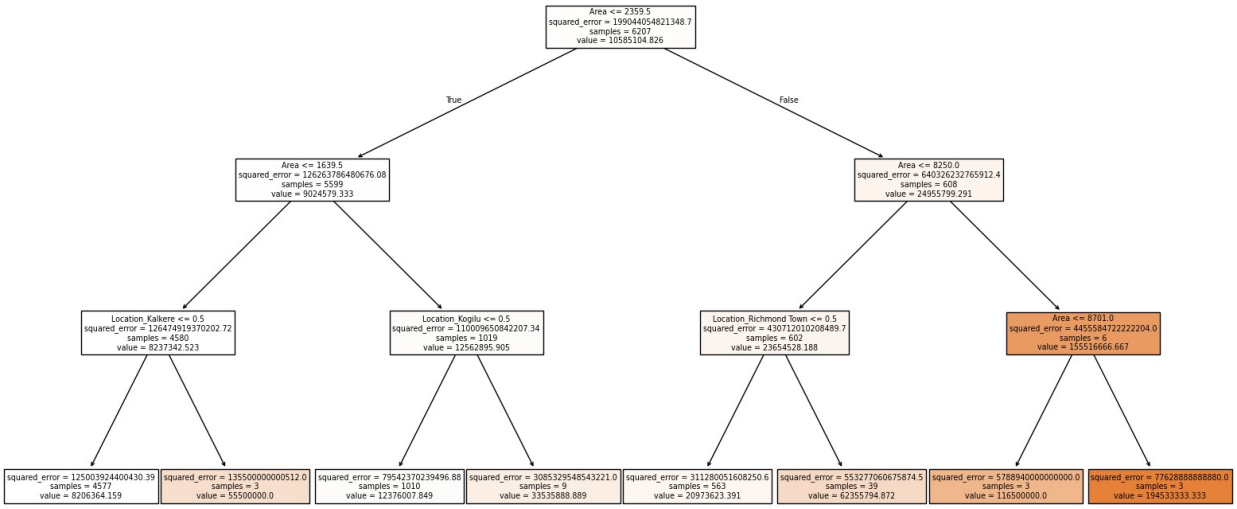
```
import pandas as pd
from sklearn.svm import SVR
```

```
# Load dataset
df = pd.read_csv("/content/sample_data/Bangalore.csv")

# Select features and target
X = df[['Area', 'No. of Bedrooms']]
y = df['Price']

# Build and train SVR model
model = SVR(kernel="linear")
model.fit(X, y)

# Prediction
y_pred = model.predict(X)


print("Predictions:", y_pred)
```

```
Predictions: [18498520.73149357  5553688.4184129
6309526.68504647 ...
  6106465.65818969  9135305.2342839   6213636.75569747]
```

Question 5: Build Decision Tree Regressor to predict Sale price of the house.

```
from sklearn.tree import DecisionTreeRegressor, plot_tree
import matplotlib.pyplot as plt
import pandas as pd

X = df[['Area','No. of Bedrooms', 'Location']]
y = df['Price']
X = pd.get_dummies(X, columns=['Location'], drop_first=True)

dt = DecisionTreeRegressor(max_depth=3, random_state=42)
dt.fit(X, y)

plt.figure(figsize=(20, 10))
plot_tree(dt, filled=True, feature_names=X.columns)
plt.show()
```



Question 6: Build Random Forest Regression model to predict Sale price of the house.

```
from sklearn.ensemble import RandomForestRegressor # Changed to
Regressor
import matplotlib.pyplot as plt
import pandas as pd

X = df[['Area','No. of Bedrooms', 'Location']]
y = df['Price']
X = pd.get_dummies(X, columns=['Location'], drop_first=True)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X, y)

y_pred = rf.predict(X)

print(y_pred)


[29883196.66666667  7807448.46153846  4957118.68253968 ...
  8585327.6425      5350171.33333333  8444525.0131746 ]

df1 =pd.read_csv("/content/sample_data/BankChurners.csv")
print(df1.value_counts())

CLIENTNUM  Attrition_Flag     Customer_Age  Gender  Dependent_count
Education_Level  Marital_Status  Income_Category  Card_Category
Months_on_book  Total_Relationship_Count  Months_Inactive_12_mon
Contacts_Count_12_mon  Credit_Limit  Total_Revolving_Bal
Avg_Open_To_Buy  Total_Amt_Chng_Q4_Q1  Total_Trans_Amt  Total_Trans_Ct
Total_Ct_Chng_Q4_Q1  Avg_Utilization_Ratio
Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_Count_12_
```

```
mon_Dependent_count_Education_Level_Months_Inactive_12_mon_1
Naive_Bayes_Classifier_Attrition_Flag_Card_Category_Contacts_Count_12_
mon_Dependent_count_Education_Level_Months_Inactive_12_mon_2
828343083  Existing Customer  43            F       4
High School      Unknown         Less than $40K   Blue            39
3                        3                       3
2786.0        1793                993.0           0.803
3646             68              0.659               0.644
0.000308
0.999690
1
708082083  Existing Customer  45            F       3
High School      Married         Less than $40K   Blue            36
4                        3                       3
3544.0        1661                1883.0          0.831
15149            111             0.734               0.469
0.000305
0.999690
1
708083283  Attrited Customer  58            M       0
Unknown          Single          $40K - $60K      Blue            45
3                        1                       3
3421.0        2517                904.0           0.992
992              21              0.400               0.736
0.988690
0.011310
1
708084558  Attrited Customer  46            M       3
Doctorate        Divorced        $80K - $120K     Blue            38
6                        3                       3
8258.0        1771                6487.0          0.000
1447             23              0.000               0.214
0.997800
0.002196
1
708085458  Existing Customer  34            F       2
Uneducated       Single          Less than $40K   Blue            24
6                        2                       2
1438.3        0                   1438.3          0.827
3940             82              0.952               0.000
0.000117
0.999880
1

..
708128733  Existing Customer  50            F       3
Post-Graduate    Single          Unknown          Blue            36
5                        4                       3
1814.0        0                   1814.0          0.852
```

```
5014              99              0.623                0.000
0.000446
0.999550
1
708125733  Existing Customer  46            F       2
College          Divorced        Less than $40K   Blue            36
4                         3                        1
1438.3        0                   1438.3           0.906
4311              77              0.833                0.000
0.000105
0.999890
1
708123033  Existing Customer  41            F       3
Graduate         Single          Less than $40K   Silver          30
2                         1                        1
11463.0       0                   11463.0          0.908
14511             105             0.721                0.000
0.000031
0.999970
1
708121908  Attrited Customer  48            M       4
Unknown          Married         $80K - $120K     Blue            36
2                         3                        2
22917.0       0                   22917.0          0.574
2045              45              0.500                0.000
0.995200
0.004799
1
708119658  Existing Customer  49            F       4
Graduate         Married         $40K - $60K      Blue            38
4                         2                        1
12836.0       1034                11802.0          0.663
2519              53              0.472                0.081
0.000072
0.999930
1
Name: count, Length: 10127, dtype: int64
```

Task2 Question 1: Build a Logistic Regression classification model which will predict whether a customer is at risk to churn from the platform

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

df1['Attrition_Flag']=df1['Attrition_Flag'].map({'Existing
Customer':0, 'Attrited Customer':1})

X = df1.drop(columns=['Attrition_Flag', 'CLIENTNUM'])
```

```
y = df1['Attrition_Flag']

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = LogisticRegression(class_weight = 'balanced')
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Prediction: ", y_pred)

accuracy = accuracy_score(y_test,y_pred)
print("Accuracy Score: ", accuracy)
```

```
Prediction:  [0 0 0 ... 0 0 0]
Accuracy Score:  0.9022704837117473

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/
_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(
```

Question 2: Build Naive Bayes model which will predict whether a customer is at risk to churn from the platform

```
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
import pandas as pd

df1 = pd.read_csv("/content/sample_data/BankChurners.csv")


df1['Attrition_Flag'] = df1['Attrition_Flag'].map({'Existing
Customer':0, 'Attrited Customer':1})

X = df1.drop(columns=['Attrition_Flag', 'CLIENTNUM'])
y = df1['Attrition_Flag']

X = pd.get_dummies(X, drop_first=True)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
print("Prediction: ", y_pred)
```

```
Prediction:  [0 0 0 ... 0 0 0]
```

Question 3: Build a K-nearest classifier which will predict whether a customer is at risk to churn from the platform.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

df2 = pd.read_csv("/content/sample_data/BankChurners.csv")

df2['Attrition_Flag'] = df2['Attrition_Flag'].map({'Existing
Customer':0, 'Attrited Customer':1})

X = df2.drop(columns=['Attrition_Flag', 'CLIENTNUM'])
y = df2['Attrition_Flag']

X = pd.get_dummies(X, drop_first=True)

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)
print("Prediction: ", y_pred)
```

```
Prediction:  [0 0 0 ... 0 0 0]
```